Pydantic：実践的な例で学ぶガイド

ベクスルツ・トゥイチエフ (Bexruz Tuychiev)

2025 年 6 月 25 日

Pydantic とは何か、何に使われるのか、そして Marshmallow や Python の dataclasses のような代替手段とどのように異なるのかを学びます。

Pydantic は、型ヒントを実行時検証ルールに変換できる Python で最も人気のあるデータ検証ライブラリです。

数多くの `isinstance()` チェックやカスタム検証関数を書く代わりに、馴染みのある Python の構文を使用してデータ構造を一度定義するだけです。

残りの処理は Pydantic がすべて行います。受信データの検証、必要に応じた型の変換、そして検証失敗時の明確なエラーメッセージの提供です。

型ヒントと適切なドキュメントを備えた、このシンプルな Python 関数を考えてみてください。

```
def calculate_user_discount(age: int, is_premium_member: bool, purchase_amount: float)
     -> float:
    """Calculate discount percentage based on user profile and purchase amount."""
    if age >= 65:
        base_discount = 0.15
    elif is_premium_member:
        base_discount = 0.10
    else:
        base_discount = 0.05
    return purchase_amount * base_discount
```

そのシグネチャを見れば、両端でどのような種類のデータを受け入れ、生成するのかが正確に分かります。しかし、エンドユーザーについても同じことが言えるでしょうか？ 結局のところ、これらの型ヒントは単なるドキュメント目的のものであり、もし誰かが誤ったデータ型を渡しても、Python のインタプリタが困ることはありません。

```
# 型が完全に間違っているにもかかわらず、エラーなしで実行されます！
discount = calculate_user_discount(True, 1, 5)
print(discount) # 出力: 0.5 （Trueは1になり、1は真とみなされるため、5 * 0.10 となる）
```

これは Python の動的型付けの典型的な例です。プログラマに迅速な開発と自由をもたらしますが、その代償として、本番環境で表面化しがちな型検証の問題を引き起こすことになります。

このチュートリアルでは、実際のアプリケーションで Pydantic を使用する方法を解説します。最初のデータモデルから始めて、本番システムで使用される高度なパターンへと進みます。ネストされたデータ構造の検証方法、複雑なビジネスルール用のカスタムバリデータの記述方法、そして FastAPI や SQLAlchemy などの人気のあるフレームワークと Pydantic を統合する方法を発見できます。

1 Pydantic とは何か？

外部世界とやり取りするアプリケーションを構築するとき、あなたは実質的にデータ・ルーレットのゲームをプレイしているようなものです。ユーザーが整数ではなく文字列として電話番号を送信したり、API が数値期待場所に null 値を返したり、設定ファイルにタイポが含まれていて午前 3 時にアプリケーションがクラッシュしたりすることがあります。

この問題に対する従来の Python のアプローチは、すぐに手に負えなくなります。最終的には次のような検証コードを書くことになります。

```
def create_user(data):
    # 手動検証の悪夢
    if not isinstance(data.get('age'), int):
        raise ValueError("Age must be an integer")
    if data['age'] < 0 or data['age'] > 150:
        raise ValueError("Age must be between 0 and 150")
```

```
    if not isinstance(data.get('email'), str) or '@' not in data['email']:
        raise ValueError("Invalid email format")
    if not isinstance(data.get('is_active'), bool):
        raise ValueError("is_active must be a boolean")

    # 最終的にユーザーを作成...
    return User(data['age'], data['email'], data['is_active'])
```

これをアプリケーション内のすべてのデータ構造に掛け合わせると、ビジネスロジックよりも検証コードを書くことに多くの時間を費やすことになります。

Pydantic は、型ヒント、実行時検証、自動シリアル化という 3 つの強力な概念を組み合わせることで、この問題を解決します。手動でのチェックの代わりに、Python の型アノテーション構文を使用してデータ構造を一度定義するだけで、Pydantic がすべての検証を自動的に処理します。

```
from pydantic import BaseModel, EmailStr
from typing import Optional

class User(BaseModel):
    age: int
    email: EmailStr
    is_active: bool = True
    nickname: Optional[str] = None

# Pydanticはデータを自動的に検証および変換します
user_data = {
    "age": "25", # 文字列がintに変換されます
    "email": "john@example.com",
    "is_active": "true" # 文字列がboolに変換されます
}

user = User(**user_data)
print(user.age) # 25（整数として）
print(user.model_dump()) # クリーンな辞書出力
```

1.1 Pydantic の特徴

Pydantic のアプローチにはいくつかの利点があります。最初に気づくメリットはパフォーマンスです。Pydantic のコア検証ロジックは Rust で書かれており、ほとんどの場合において手書きの Python 検証よりも高速です。検証がボトルネックになることなく、アプリケーションは何千ものリクエストを処理できます。

また、Pydantic は現代のフレームワークとの統合も可能です。Python で最も急成長している Web フレームワークの 1 つである FastAPI は、Pydantic モデルを使用して API ドキュメントを自動生成し、リクエストボディを検証し、レスポンスをシリアル化します。Pydantic モデルを定義すると、OpenAPI スキーマの生成が無料（自動）で手に入ります。

```python
from fastapi import FastAPI
from pydantic import BaseModel, EmailStr

app = FastAPI()

class UserCreate(BaseModel):
    name: str
    email: EmailStr
    age: int

@app.post("/users/")
async def create_user(user: UserCreate):
    # FastAPIはリクエストボディを自動的に検証し、
    # PydanticモデルからAPIドキュメントを生成します
    return {"message": f"Created user {user.name}"}
```

JSON スキーマの生成は、すべての Pydantic モデルで自動的に行われます。これは、データ構造が自己ドキュメント化されることを意味し、同じ信頼できる情報源（シングル・ソース・オブ・トゥルース）から、クライアントライブラリ、フロントエンドアプリケーション用の検証ルール、またはデータベーススキーマを生成できるようになります。

2 Pydantic vs. Dataclasses vs. Marshmallow

しかし、どのような場合に代替手段ではなく Pydantic を選ぶべきでしょうか？ Pydantic と dataclasses を比較する場合、決定打となるのは検証の必要性です。Python の `@dataclass` は入力を信頼できるシンプルなデータコンテナに最適ですが、検証、シリアル化、および Web フレームワークとの統合が必要な場合には Pydantic が優れています。

```python
from dataclasses import dataclass
from pydantic import BaseModel

# Dataclass: 高速、シンプル、検証なし
@dataclass
class UserDataclass:
    name: str
    age: int

# Pydantic: 検証、シリアル化、フレームワーク統合
class UserPydantic(BaseModel):
    name: str
    age: int
```

Pydantic と Marshmallow を比較すると、Pydantic の型ヒントアプローチの方が Python 開発者にとって自然に感じられます。一方、Marshmallow はスキーマクラスを使用します。また、Pydantic はより優れたパフォーマンスと、現代の非同期フレームワークとの簡単な統合を提供します。

Pydantic は、API の構築、外部データの処理、設定の管理、あるいはデータ検証の失敗を後々謎のバグを引き起こす原因にするのではなく、早期に検出すべきあらゆるシナリオにおいて最適に機能します。実行時エラーを、開発者とユーザーの両方が何が問題だったのかを理解するのに役立つ、明確で実用的な検証メッセージに変換します。

3　Pydantic を始めよう

インストールから最初のデータモデルの作成まで、プロジェクトで Pydantic を使用する方法を詳しく見ていきましょう。

3.1　インストールとセットアップ

Pydantic のセットアップは簡単ですが、初心者がハマりがちな共通の落とし穴がいくつかあります。適切なインストールプロセスを順を追って説明し、不要なデバッグを引き起こすミスを回避できるようにサポートします。

まず、プロジェクトの依存関係を隔離するために仮想環境を作成します。これにより、後々謎のインポートエラーを引き起こすバージョン競合を防ぐことができます。

```
# 仮想環境を作成して有効化する（zshおよびbashで動作）
python -m venv pydantic_env
source pydantic_env/bin/activate # macOS/Linuxの場合

# pydantic_env\Scripts\activate # Windowsの場合
```

次に、以下のコマンドで Pydantic をインストールします。

```
pip install pydantic
```

実際のアプリケーション（およびこのチュートリアル）でおそらく必要になるメールアドレス検証機能のために、メール拡張機能もインストールします。

```
pip install "pydantic[email]"
```

`pydantic[email]` のまわりのクォーテーション（引用符）は、ほとんどすべてのターミナルで重要です。これがないと、”no matches found”（一致するものがありません）というエラーが表示されることがあります。

注意: `ModuleNotFoundError: No module named 'pydantic'` が発生した場合は、次の点を確認してください。

- **不適切な Python 環境:** 仮想環境が有効化されていることを確認してください。
- **複数の Python バージョン:** Pydantic をインストールしたのと同じ Python バージョンを使用していることを確認してください。
- **IDE 設定:** コードエディタが別の Python インタプリタを使用している可能性があります。

重要な命名に関する警告: Python ファイルに `pydantic.py` という名前を絶対につけないでください。これにより循環インポートが発生し、混乱を招くエラーメッセージでコードが破綻します。Python は実際の Pydantic ライブラリではなく、あなたのファイルをインポートしようとしてしまいます。

3.2 最初の Pydantic モデル

実際に Pydantic の動作を確認するために、シンプルなユーザーモデルを構築してみましょう。抽象的な例から始めるのではなく、Web フォームからのユーザー登録データの検証という現実的な問題を解決します。

```
from pydantic import BaseModel, EmailStr
from typing import Optional
from datetime import datetime

class User(BaseModel):
    name: str
    email: EmailStr
    age: int
    is_active: bool = True
    created_at: Optional[datetime] = None

# クリーンなデータでテスト
clean_data = {
    "name": "Alice Johnson",
```

```
    "email": "alice@example.com",
    "age": 28
}

user = User(**clean_data)
print(f"User created: {user.name}, Age: {user.age}")
print(f"Model output: {user.model_dump()}")
```

出力:

```
User created: Alice Johnson, Age: 28
Model output: {'name': 'Alice Johnson', 'email': 'alice@example.com', 'age': 28, '
    is_active': True, 'created_at': None}
```

このモデル定義で何が行われているのかを分解して説明します。

- **Pydantic モデルの作成:** あなたの `User` クラスは `BaseModel` を継承しており、これにより Pydantic のすべての検証およびシリアル化機能が備わります。この継承により、通常の Python クラスがデータ検証ツールに変わります。
- **フィールドの定義:** クラス内の各行は、予期される型を持つフィールドを定義します。`name: str` という構文は Pydantic に対し `name` が文字列であるべきことを伝え、`age: int` は `age` が整数であるべきことを意味します。
- **EmailStr の解説:** `EmailStr` は、メールアドレスを自動的に検証する特別な Pydantic 型です。先ほどインストールした `pydantic[email]` パッケージから提供され、正規表現を使用してメールの形式が有効であることを保証します。もし誰かが `"not-an-email"` を渡すと、Pydantic は検証エラーを発生させます。
- **デフォルト値:** `is_active: bool = True` のようなフィールドにはデフォルト値があります。ユーザー作成時にこれらのフィールドを提供しない場合、Pydantic はデフォルト値を使用します。また、`created_at` の `= None` はこのフィールドをオプショナル（任意）にします。
- **モデルのインスタンス化:** `User(**clean_data)` を呼び出すと、`**` が辞書を展開し、各キーと値のペアをキーワード引数としてモデルコンストラクタに渡します。

次に、Pydantic の自動型変換が実際に動作する様子を見てみましょう。

```
# うまく処理される少し散らかったデータ
messy_data = {
    "name": "Bob Smith",
    "email": "bob@company.com",
    "age": "35", # intではなく文字列
    "is_active": "true" # boolではなく文字列
}

user = User(**messy_data)
print(f"Age type: {type(user.age)}") # <class 'int'>
print(f"Is active type: {type(user.is_active)}") # <class 'bool'>
```

出力:

```
Age type: <class 'int'>
Is active type: <class 'bool'>
```

ご覧のとおり、age および is_active フィールドは検証を通じて適切な形式に自動変換されています。

4　まとめ

これで、Pydantic を使用することで手動で検証コードを書く退屈な作業から解放されることがお分かりいただけたでしょう。関数を isinstance チェックや手動の型変換でごちゃ混ぜにする代わりに、データ構造を一度定義するだけで、面倒な作業はすべて Pydantic に任せることができます。